## EXERCISE

1. What is NumPy ? How to install it?
2. What is an array and how is it different from a list? What is the name of the built-in array class in NumPy ?
3. What do you understand by rank of an ndarray?
4. Create the following NumPy arrays:
   a) A 1-D array called `zeros` having 10 elements and all the elements are set to zero.
   b) A 1-D array called `vowels` having the elements 'a', 'e', 'i', 'o' and 'u'.
   c) A 2-D array called ones having 2 rows and 5 columns and all the elements are set to 1 and dtype as int.
   d) Use nested Python lists to create a 2-D array called `myarray1` having 3 rows and 3 columns and store the following data:

      2.7, -2, -19
      0, 3.4, 99.9
      10.6, 0, 13

   e) A 2-D array called `myarray2` using `arange()` having 3 rows and 5 columns with start value = 4, step size 4 and dtype as float.
5. Using the arrays created in Question 4 above, write NumPy commands for the following:
   a) Find the dimensions, shape, size, data type of the items and itemsize of arrays `zeros`, `vowels`, `ones`, `myarray1` and `myarray2`.
   b) Reshape the array ones to have all the 10 elements in a single row.
   c) Display the 2nd and 3rd element of the array `vowels`.
   d) Display all elements in the 2nd and 3rd row of the array `myarray1`.
   e) Display the elements in the 1st and 2nd column of the array `myarray1`.
   f) Display the elements in the 1st column of the 2nd and 3rd row of the array `myarray1`.
   g) Reverse the array of `vowels`.
6. Using the arrays created in Question 4 above, write NumPy commands for the following:

a) Divide all elements of array ones by 3.

b) Add the arrays `myarray1` and `myarray2`.

c) Subtract `myarray1` from `myarray2` and store the result in a new array.

d) Multiply `myarray1` and `myarray2` elementwise.

e) Do the matrix multiplication of `myarray1` and `myarray2` and store the result in a new array `myarray3`.

f) Divide `myarray1` by `myarray2`.

g) Find the cube of all elements of `myarray1` and divide the resulting array by 2.

h) Find the square root of all elements of `myarray2` and divide the resulting array by 2. The result should be rounded to two places of decimals.

7. Using the arrays created in Question 4 above, write NumPy commands for the following:

   a) Find the transpose of `ones` and `myarray2`.

   b) Sort the array `vowels` in reverse.

   c) Sort the array `myarray1` such that it brings the lowest value of the column in the first row and so on.

8. Using the arrays created in Question 4 above, write NumPy commands for the following:

   a) Use `NumPy. split()` to split the array `myarray2` into 5 arrays columnwise. Store your resulting arrays in `myarray2A, myarray2B, myarray2C, myarray2D` and `myarray2E`. Print the arrays `myarray2A, myarray2B, myarray2C, myarray2D` and `myarray2E`.

   b) Split the array `zeros` at array index 2, 5, 7, 8 and store the resulting arrays in `zerosA, zerosB, zerosC` and `zerosD` and print them.

   c) Concatenate the arrays `myarray2A, myarray2B` and `myarray2C` into an array having 3 rows and 3 columns.

9. Create a 2-D array called `myarray4` using `arange()` having 14 rows and 3 columns with start value = -1, step size 0.25 having. Split this array row wise into 3 equal parts and print the result.

10. Using the `myarray4` created in the above questions, write commands for the following:

    a) Find the sum of all elements.

    b) Find the sum of all elements row wise.

c) Find the sum of all elements column wise.

d) Find the max of all elements.

e) Find the min of all elements in each row.

f) Find the mean of all elements in each row.

g) Find the standard deviation column wise.

## CASE STUDY (SOLVED)

We have already learnt that a data set (or dataset) is a collection of data. Usually a data set corresponds to the contents of a database table, or a statistical data matrix, where every column of the table represents a particular variable, and each row corresponds to a member or an item etc. A data set lists values for each of the variables, such as height and weight of a student, for each row (item) of the data set. Open data refers to information released in a publicly accessible repository.

The Iris flower data set is an example of an open data. It is also called Fisher's Iris data set as this data set was introduced by the British statistician and biologist Ronald Fisher in 1936. The Iris data set consists of 50 samples from each of the three species of the flower Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured for each sample: the length and the width of the sepals and petals, in centimeters. Based on the combination of these four features, Fisher developed a model to distinguish one species from each other. The full data set is freely available on UCI Machine Learning Repository at https://archive.ics.uci.edu/ml/datasets/iris.

We shall use the following smaller section of this data set having 30 rows (10 rows for each of the three species). We shall include a column for species number that has a value 1 for Iris setosa, 2 for Iris virginica and 3 for Iris versicolor.

| Sepal Length | Sepal Width | Petal Length | Petal Width | Iris | Species No |
|---|---|---|---|---|---|
| 5.1 | 3.5 | 1.4 | 0.2 | Iris-setosa | 1 |
| 4.9 | 3 | 1.4 | 0.2 | Iris-setosa | 1 |
| 4.7 | 3.2 | 1.3 | 0.2 | Iris-setosa | 1 |
| 4.6 | 3.1 | 1.5 | 0.2 | Iris-setosa | 1 |
| 5 | 3.6 | 1.4 | 0.2 | Iris-setosa | 1 |
| 5.4 | 3.9 | 1.7 | 0.4 | Iris-setosa | 1 |
| 4.6 | 3.4 | 1.4 | 0.3 | Iris-setosa | 1 |
| 5 | 3.4 | 1.5 | 0.2 | Iris-setosa | 1 |
| 4.4 | 2.9 | 1.4 | 0.2 | Iris-setosa | 1 |

| | | | | | |
|---|---|---|---|---|---|
| 4.9 | 3.1 | 1.5 | 0.1 | Iris-setosa | 1 |
| 5.5 | 2.6 | 4.4 | 1.2 | Iris-versicolor | 2 |
| 6.1 | 3 | 4.6 | 1.4 | Iris-versicolor | 2 |
| 5.8 | 2.6 | 4 | 1.2 | Iris-versicolor | 2 |
| 5 | 2.3 | 3.3 | 1 | Iris-versicolor | 2 |
| 5.6 | 2.7 | 4.2 | 1.3 | Iris-versicolor | 2 |
| 5.7 | 3 | 4.2 | 1.2 | Iris-versicolor | 2 |
| 5.7 | 2.9 | 4.2 | 1.3 | Iris-versicolor | 2 |
| 6.2 | 2.9 | 4.3 | 1.3 | Iris-versicolor | 2 |
| 5.1 | 2.5 | 3 | 1.1 | Iris-versicolor | 2 |
| 5.7 | 2.8 | 4.1 | 1.3 | Iris-versicolor | 2 |
| 6.9 | 3.1 | 5.4 | 2.1 | Iris-virginica | 3 |
| 6.7 | 3.1 | 5.6 | 2.4 | Iris-virginica | 3 |
| 6.9 | 3.1 | 5.1 | 2.3 | Iris-virginica | 3 |
| 5.8 | 2.7 | 5.1 | 1.9 | Iris-virginica | 3 |
| 6.8 | 3.2 | 5.9 | 2.3 | Iris-virginica | 3 |
| 6.7 | 3.3 | 5.7 | 2.5 | Iris-virginica | 3 |
| 6.7 | 3 | 5.2 | 2.3 | Iris-virginica | 3 |
| 6.3 | 2.5 | 5 | 1.9 | Iris-virginica | 3 |
| 6.5 | 3 | 5.2 | 2 | Iris-virginica | 3 |
| 6.2 | 3.4 | 5.4 | 2.3 | Iris-virginica | 3 |

You may type this using any text editor `(Notepad, gEdit or any other)` in the way as shown below and store the file with a name called `Iris.txt`. (In case you wish to work with the entire dataset you could download a .csv file for the same from the Internet and `save it as Iris.txt)`. The headers are:

sepal length, sepal width, petal length, petal width, iris, Species No

5.1, 3.5, 1.4, 0.2, Iris-setosa, 1

4.9, 3, 1.4, 0.2, Iris-setosa, 1

4.7, 3.2, 1.3, 0.2, Iris-setosa, 1

4.6, 3.1, 1.5, 0.2, Iris-setosa, 1

5, 3.6, 1.4, 0.2, Iris-setosa, 1

5.4, 3.9, 1.7, 0.4, Iris-setosa, 1

4.6, 3.4, 1.4, 0.3, Iris-setosa, 1

5, 3.4, 1.5, 0.2, Iris-setosa, 1

4.4, 2.9, 1.4, 0.2, Iris-setosa, 1

4.9, 3.1, 1.5, 0.1, Iris-setosa, 1

5.5, 2.6, 4.4, 1.2, Iris-versicolor, 2

6.1, 3, 4.6, 1.4, Iris-versicolor, 2

5.8, 2.6, 4, 1.2, Iris-versicolor, 2

5, 2.3, 3.3, 1, Iris-versicolor, 2

5.6, 2.7, 4.2, 1.3, Iris-versicolor, 2

5.7, 3, 4.2, 1.2, Iris-versicolor, 2

5.7, 2.9, 4.2, 1.3, Iris-versicolor, 2

6.2, 2.9, 4.3, 1.3, Iris-versicolor, 2

5.1, 2.5, 3, 1.1, Iris-versicolor, 2

5.7, 2.8, 4.1, 1.3, Iris-versicolor, 2

6.9, 3.1, 5.4, 2.1, Iris-virginica, 3

6.7, 3.1, 5.6, 2.4, Iris-virginica, 3

6.9, 3.1, 5.1, 2.3, Iris-virginica, 3

5.8, 2.7, 5.1, 1.9, Iris-virginica, 3

6.8, 3.2, 5.9, 2.3, Iris-virginica, 3

6.7, 3.3, 5.7, 2.5, Iris-virginica, 3

6.7, 3, 5.2, 2.3, Iris-virginica, 3

6.3, 2.5, 5, 1.9, Iris-virginica, 3

6.5 , 3, 5.2, 2, Iris-virginica, 3

6.2, 3.4, 5.4, 2.3, Iris-virginica, 3

1. Load the data in the file `Iris.txt` in a 2-D array called `iris`.
2. Drop column whose index = 4 from the array `iris`.
3. Display the shape, dimensions and size of `iris`.
4. Split `iris` into three 2-D arrays, each array for a different species. Call them `iris1, iris2, iris3`.
5. Print the three arrays `iris1, iris2, iris3`
6. Create a 1-D array `header` having elements "sepal length", "sepal width", "petal length", "petal width", "Species No" in that order.
7. Display the array `header`.
8. Find the max, min, mean and standard deviation for the columns of the iris and store the results in the arrays `iris_max, iris_min, iris_avg, iris_std, iris_var` respectively. The results must be rounded to not more than two decimal places.

9. Similarly find the max, min, mean and standard deviation for the columns of the `iris1`, `iris2` and `iris3` and store the results in the arrays with appropriate names.
10. Check the minimum value for sepal length, sepal width, petal length and petal width of the three species in comparison to the minimum value of sepal length, sepal width, petal length and petal width for the data set as a whole and fill the table below with True if the species value is greater than the dataset value and False otherwise.

| | Iris setosa | Iris virginica | Iris versicolor |
|---|---|---|---|
| sepal length | | | |
| sepal width | | | |
| petal length | | | |
| petal width | | | |

11. Compare Iris setosa's average sepal width to that of Iris virginica.
12. Compare Iris setosa's average petal length to that of Iris virginica.
13. Compare Iris setosa's average petal width to that of Iris virginica.
14. Save the array `iris_avg` in a comma separated file named `IrisMeanValues.txt` on the hard disk.
15. Save the arrays `iris_max`, `iris_avg`, `iris_min` in a comma separated file named `IrisStat.txt` on the hard disk.